

ESCUELA DE EDUCACIÓN SECUNDARIA TÉCNICA N.º 2 "EDUCACIÓN Y TRABAJO"
ESPECIALIDAD INFORMÁTICA / PROGRAMACIÓN

DOCUMENTO DE ARQUITECTURA DE SOFTWARE (SAD)

Modelo de Vistas 4+1 (IEEE Std 1471 / ISO 42010) — Patrón Clean Architecture + Feature-First

Líder de Arquitectura & QA Lead:	Alan Martinez
Equipo de Desarrollo:	Ivan Ismael Cardozo, Alex Heredia, Marcos Silvani, Franco Sarraute
Estándar y Patrón:	Modelo 4+1 Kruchten — Clean Architecture + Feature-First & Repository Pattern
Fecha y Versión:	2026-08-21 Versión 1.0 (Congelamiento de Arquitectura)

1. Visión General de la Arquitectura y Selección del Patrón

El Documento de Arquitectura de Software (Software Architecture Document - SAD) formaliza la estructura general de la Plataforma Inteligente Hospitalaria para la Notificación Crítica de Código Azul. La solución adopta el Modelo de Vistas 4+1 de Philippe Kruchten bajo las pautas de las normas IEEE Std 1471 / ISO 42010.

1.1. Justificación del Patrón: Clean Architecture + Feature-First

Para conciliar la alta profesionalidad requerida con la agilidad necesaria para la competencia, se selecciona el patrón Clean Architecture adaptado a una estructura Feature-First con Repository Pattern. Esta decisión descarta deliberadamente la arquitectura Hexagonal pura debido a la sobrecarga de adaptadores que introduciría en un equipo de 5 integrantes.

Decisión Arquitectónica Clave (ADR-01):

La arquitectura Feature-First divide el proyecto por características independientes (Auth, Código Azul, Auditoría). Esto permite que Ivan Cardozo (Backend) y Alex Heredia (Tiempo Real) desarrollen en paralelo dentro de directorios aislados sin generar conflictos de fusión en Git.

2. Vistas de la Arquitectura (Modelo 4+1)

2.1. Vista Lógica (Estructura Interna del Backend)

La lógica de negocio del servidor Node.js se organiza en capas concéntricas con regla de dependencia hacia adentro:

- **Capa 1 (Domain / Core):** Entidades del dominio clínico, modelos transaccionales y tipos ENUM de PostgreSQL.
- **Capa 2 (Use Cases / Application):** Casos de uso independientes (ActivarCodigoAzul, ConfirmarACK, CancelarAlerta).
- **Capa 3 (Data / Infrastructure):** Implementación del Repository Pattern para abstracción de consultas SQL.
- **Capa 4 (Presentation / Drivers):** Controladores API REST (Express/NestJS) y Gateways de WebSockets (Socket.IO).

2.2. Vista de Procesos (Flujo Asíncrono y Tiempo Real)

Describe el comportamiento reactivo e interacciones de concurrencia ante la activación de una emergencia:

- **Paso 1 (Gateway):** Request HTTP POST entrante desde cliente Web o App Móvil.
- **Paso 2 (Persistencia):** El backend valida el token JWT e inicia una transacción ACID en PostgreSQL con nivel de aislamiento READ COMMITTED.
- **Paso 3 (Broadcast WebSockets):** Emisión broadcast asíncrona mediante Socket.IO a través del Event Loop no bloqueante de Node.js.
- **Paso 4 (Push Notification):** Despacho asíncrono a la API de Firebase Cloud Messaging (FCM) para notificaciones Push en segundo plano.

2.3. Vista de Desarrollo (Estructura de Directorios del Repositorio)

Organización oficial del código fuente para el equipo:

```
src/
├── core/                # Middlewares JWT, DB Config, Helpers
├── features/
│   ├── auth/           # Autenticación y RBAC
│   └── codigo_azul/    # Módulo Crítico Hospitalario
│       ├── domain/    # Entidades e Interfaces de Repositorio
│       ├── use_cases/ # Lógica pura de Casos de Uso
│       ├── data/      # Consultas SQL PostgreSQL / Mappers
│       └── presentation/ # Express Controllers & Socket.IO Gateway
```

2.4. Vista Física y de Despliegue (Modelo C4 / Nodos)

Mapeo de la infraestructura distribuida:

- **Nodo 1 (Cliente Web):** Browsers navegadores ejecutando SPA en React (Dashboard de Guardia).
- **Nodo 2 (Cliente Móvil):** Dispositivos Android ejecutando la App en React Native con servicio background FCM.
- **Nodo 3 (Backend Server):** Servidor Central Node.js hospedando REST API y Socket.IO WebSocket Server.
- **Nodo 4 (Database Server):** Motor de base de datos relacional PostgreSQL garantizando integridad transaccional ACID.

3. Tabla Matriz del Stack Tecnológico Definitivo (Congelado)

Capa de Arquitectura	Tecnología Seleccionada	Justificación Técnica y Operativa
Backend Core	Node.js + Express (o NestJS)	Event Loop no bloqueante ideal para I/O intensivo; comparte lenguaje JS/TS con frontend.
Tiempo Real (WebSockets)	Socket.IO	Canal bidireccional de baja latencia con fallback automático y emisión broadcast en milisegundos.
Notificaciones Push	Firebase Cloud Messaging (FCM)	Garantiza la entrega de alertas sonoras cuando los dispositivos Android están bloqueados o en background.
Base de Datos Relacional	PostgreSQL	Garantías ACID plenas, soporte nativo de tipos ENUM para estados y JSONB para auditoría de eventos.
Frontend Web Dashboard	React SPA	Actualización reactiva en tiempo real de la interfaz sin necesidad de recargar la página.
Aplicación Móvil	React Native	Rendimiento nativo en Android, reutilización de lógica con React Web y fácil consumo de WebSockets/FCM.

4. Referencias Normativas (APA 7.ª Edición)

Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4.ª ed.). Addison-Wesley Professional.

IEEE Computer Society. (2000). IEEE Recommended Practice for Architectural Description of Software-Intensive Systems (IEEE Std 1471-2000 / ISO/IEC 42010). IEEE.

Martin, R. C. (2018). Clean Architecture: A Craftsman's Guide to Software Structure and Design (1.ª ed.). Prentice Hall.